GO START · ПРАКТИЧЕСКИЙ КУРС

Урок 1

Основы языка Go

Полное объяснение с нуля и две практические работы

Модули · пакеты · типы · ввод · условия · циклы · функции · ошибки · Git

Как работать с этим уроком

Этот материал рассчитан на изучение с нуля. Не требуется заранее знать синтаксис другого языка. Цель урока - не запомнить все конструкции, а понять, из каких частей состоит программа и как данные проходят через нее.

Рекомендуемый порядок

1. Прочитай раздел, не набирая код. Сначала постарайся понять идею.
2. Набери короткие примеры вручную в отдельном временном файле. Не копируй их механически.
3. Измени числа, строки и условия. Перед запуском предположи результат.
4. Ответь на вопросы самопроверки без подсказок.
5. Только после этого переходи к практическим заданиям.

Главное правило
Если конструкция непонятна, не пытайся компенсировать это запоминанием. Остановись и сформулируй конкретный вопрос: что хранится в переменной, когда вызывается функция, какое значение возвращается, почему выполняется эта ветка.

Результат урока

- Объяснить разницу между модулем, пакетом, файлом и функцией.
- Самостоятельно создать небольшую исполняемую программу.
- Объявлять переменные и константы, выбирать подходящий базовый тип.
- Читать данные, проверять ошибку и валидировать значение.
- Использовать if, switch и for.
- Разделять программу на функции и понимать возвращаемые значения.
- Пройти базовый цикл Git от рабочей директории до push.

Содержание

1. Как Go выполняет программу · 2. Модуль и пакет · 3. Первая программа · 4. Синтаксис · 5. Типы и переменные · 6. Константы и операции · 7. Вывод · 8. Ввод · 9. Ошибки · 10. Условия · 11. switch · 12. Циклы · 13. Функции · 14. Область видимости · 15. Деньги · 16. Отладка · 17. Практика · 18. Git · 19. Самопроверка

1. Как Go выполняет программу

Программа начинается как обычный текст в файлах с расширением .go. Процессор не умеет выполнять такой текст напрямую. Инструменты Go читают исходный код, проверяют его и превращают в инструкции, понятные компьютеру.

Путь от текста до результата

Шаг	Что происходит
1. Исходный код	Ты описываешь данные и действия в .go-файле.
2. Разбор	Go проверяет структуру: package, импорты, функции, выражения.
3. Проверка типов	Компилятор убеждается, что операции допустимы для выбранных типов.
4. Компиляция	Исходный код превращается в машинный код.
5. Выполнение	Операционная система запускает полученную программу.

go run и go build

```
go run ./non_project_tasks/lesson_01/number_analyzer
```

go run временно компилирует программу и сразу запускает ее. Это удобно во время обучения и разработки.

```
go build ./...
```

go build проверяет, что пакеты можно собрать. Запись ./... означает: начать с текущей директории и проверить все вложенные Go-пакеты.

Компилятор как помощник
Сообщение компилятора - не наказание, а точное указание на несогласованность в программе. Читай его сверху вниз: имя файла, номер строки, затем описание проблемы.

Три вида проблем

- Синтаксическая ошибка: текст не соответствует грамматике Go, например пропущена фигурная скобка.
- Ошибка компиляции: структура допустима, но операция невозможна, например складываются int и string.
- Логическая ошибка: программа запускается, но выдает неверный результат из-за неправильного алгоритма.

2. Модуль, пакет, файл и функция

Эти четыре понятия образуют вложенную структуру. Их нельзя считать синонимами.

Понятие	Простое объяснение	Пример
Модуль	Проект или большая его часть, управляемая файлом go.mod.	github.com/Amaym0n/go-start
Пакет	Группа Go-файлов одной директории, которые работают вместе.	package main
Файл	Текстовый файл с исходным кодом.	main.go
Функция	Именованная последовательность действий.	func main()

Модуль

В корне модуля находится файл go.mod. Он хранит имя модуля, версию Go и позднее - сведения о внешних зависимостях.

```
go mod init github.com/Amaym0n/go-start
```

Эту команду выполняют один раз в корне нового проекта. После нее инструменты Go понимают границы модуля.

Пакет

Каждый Go-файл принадлежит ровно одному пакету. Обычно все Go-файлы одной директории имеют одинаковое имя пакета. Они могут использовать объявленные друг в друге функции и типы как части одного целого.

```
order_calculator/
main.go package main
price.go package main
discount.go package main
```

Почему директория важна
Пакет определяется не только словом после package, но и расположением файлов. Две разные директории могут иметь пакеты с одинаковым именем, но это будут разные пакеты.

Пакет main особенный: он сообщает Go, что пакет должен собираться в исполняемую программу. В нем должна существовать функция main.

Что значит «первая значимая строка»

Декларация пакета должна быть первой конструкцией с кодом в каждом Go-файле. До нее разрешены только пустые строки и комментарии. Пустая строка ничего не выполняет, а комментарий предназначен для человека, поэтому они не считаются значимым кодом.

Допустимо

```
// Краткое описание файла.

package main

import "fmt"
```

Здесь первая строка содержит комментарий, но первая инструкция языка - package main. Поэтому файл корректен.

Недопустимо

```
import "fmt"

package main
```

Импорт нельзя написать до package. Компилятор еще не знает, к какому пакету относится файл.

Почему package повторяется в каждом файле

Go компилирует пакет из нескольких файлов, но каждый файл остается самостоятельной единицей исходного текста. Поэтому каждый из них явно сообщает свое имя пакета.

Файл	Первая конструкция	Результат
main.go	package main	Корректно
price.go	package main	Корректно, тот же пакет
discount.go	package calculator	Ошибка: разные пакеты в одной директории

Точная формулировка
package - это не обязательно физически первая строка файла. Это первый элемент кода после необязательных комментариев и пустых строк.

3. Первая программа: разбор по строкам

```
package main

import "fmt"

func main() {
fmt.Println("Привет!")
}
```

package main

Файл принадлежит пакету main. Такой пакет можно превратить в исполняемую программу.

import "fmt"

Программа подключает стандартный пакет fmt. В нем находятся функции для форматированного ввода и вывода. Импорт дает доступ к экспортированным именам пакета через точку: fmt.Println.

func main()

Слово func начинает объявление функции. main - имя функции. Пустые круглые скобки означают, что она не принимает параметров. Фигурные скобки ограничивают тело функции.

fmt.Println(...)

Это вызов функции Println из пакета fmt. Значение в круглых скобках передается функции как аргумент. Здесь аргументом является строка "Привет!".

Порядок выполнения

1. Операционная система запускает программу.
2. Go начинает выполнение с функции main пакета main.
3. Вызывается fmt.Println.
4. Текст выводится в терминал.
5. В main больше нет инструкций, поэтому программа завершается.

Объявление и вызов
func main() { ... } объявляет функцию, то есть описывает ее. fmt.Println(...) вызывает уже объявленную функцию, то есть просит выполнить ее сейчас.

4. Базовые правила синтаксиса

Регистр имеет значение

Имена main, Main и MAIN различаются. Функция запуска должна называться именно main строчными буквами.

Фигурные скобки образуют блок

```
if ready {
fmt.Println("Можно продолжать")
}
```

Все инструкции между { и } относятся к условию. В Go открывающая скобка обычно находится в той же строке, что и if, for, switch или объявление функции.

Точка с запятой обычно не пишется

Формально язык использует точки с запятой, но лексический анализатор вставляет большинство из них автоматически в конце строк. Поэтому обычный Go-код пишется без явных ;.

Комментарии

```
// Комментарий до конца строки.

/* Комментарий
на несколько строк. */
```

Комментарий объясняет причину или неочевидное решение. Не нужно комментировать каждую очевидную операцию.

Форматирование

```
gofmt -w .
```

gofmt автоматически расставляет стандартные отступы и пробелы. Стиль форматирования не обсуждается внутри команды: для всех Go-проектов используется единый инструмент.

Частая ошибка новичка
Не выравнивай код пробелами вручную и не спорь с gofmt. Сосредоточься на структуре программы.

Неиспользуемые элементы

Компилятор не разрешает неиспользуемые локальные переменные и импорты. Это помогает удалять мусор и обнаруживать незавершенную логику.

5. Значения и типы

Значение - конкретные данные: число 7, строка "paid" или логическое true. Тип определяет, какие значения допустимы и какие операции с ними можно выполнять.

Тип	Что хранит	Примеры
int	Целые числа	0, 15, -3
string	Последовательность символов	"Apple", "STARTGO"
bool	Логическое значение	true, false
error	Информацию об ошибке	nil или значение ошибки

Почему типы важны

```
quantity := 3
product := "card"
```

quantity имеет тип int, a product - string. Выражение quantity + product не имеет смысла для Go: правила сложения числа и текста не определены.

Строковые литералы

Текст в двойных кавычках является строкой. Кавычки отмечают границы значения и не входят в сам текст.

```
name := "Gift card"
```

Логические значения

bool имеет ровно два значения. Они часто появляются как результат сравнения:

```
allowed := quantity <= 10
// allowed будет true или false
```

Статическая типизация

Тип определяется во время компиляции и не меняется в ходе жизни переменной. Благодаря этому многие ошибки обнаруживаются до запуска.

Тип не равен содержимому
int - это категория данных. 10 - конкретное значение этого типа. quantity - имя переменной, в которой такое значение может храниться.

6. Переменные, присваивание и нулевые значения

Переменная - именованное место для хранения значения определенного типа. Имя позволяет обратиться к значению позднее.

Объявление через var

```
var quantity int
var promo string
var paid bool
```

Если начальное значение не указано, Go записывает нулевое значение типа.

Тип	Нулевое значение
int	0
string	пустая строка ""
bool	false
error	nil

Объявление с начальным значением

```
var quantity int = 2
var title = "Code"
```

Во второй строке тип выводится из значения справа. title все равно получает конкретный тип string.

Короткое объявление :=

```
quantity := 2
title := "Code"
```

Оператор := одновременно создает локальную переменную и присваивает ей первое значение. Он используется только внутри функций.

Обычное присваивание =

```
quantity := 2
quantity = 3
```

Первая строка создает quantity. Вторая меняет уже существующее значение. Повторное quantity := 3 в том же блоке без новой переменной приведет к ошибке.

Разница в одном символе
:= создает переменную. = записывает новое значение в уже существующую переменную. Перед каждой строкой спрашивай себя: имя уже существует в этом блоке?

7. Константы, операции и преобразования

Константа

Константа обозначает значение, которое программа не должна менять. Она хорошо подходит для ограничений и фиксированных правил.

```
const maxQuantity = 10
const promoCode = "STARTGO"
```

Попытка позднее присвоить maxQuantity другое значение завершится ошибкой компиляции.

Арифметические операции

Оператор	Действие	Пример
+	Сложение	2 + 3 дает 5
-	Вычитание	7 - 2 дает 5
*	Умножение	4 * 3 дает 12
/	Деление	7 / 2 дает 3 для int
%	Остаток от деления	7 % 2 дает 1

При делении двух int дробная часть отбрасывается. Это не округление до ближайшего числа: 7 / 2 равно 3.

Сокращенные присваивания

```
total += price // то же, что total = total + price
count++ // увеличить count на 1
```

Сравнения

```
quantity == 10
quantity != 10
quantity < 10
quantity >= 1
```

Результат любого сравнения имеет тип bool.

Преобразование типов

```
small := 5
large := int64(small)
```

Go обычно не преобразует числовые типы неявно. Явная запись типа вокруг значения показывает намерение программиста.

8. Вывод данных через fmt

Чтобы использовать функции вывода, пакет fmt нужно импортировать. Имя перед точкой - пакет, имя после точки - доступная функция этого пакета.

Println

```
fmt.Println("Заказ создан")
fmt.Println("Количество:", quantity)
```

Println печатает аргументы, вставляет между ними пробелы и завершает строку переводом строки.

Printf

```
fmt.Printf("Количество: %d\n", quantity)
```

Printf использует строку-шаблон. Последующие аргументы подставляются на места спецификаторов.

Спецификатор	Ожидаемое значение
%d	Целое число
%s	Строка
%t	Логическое значение
%v	Значение в стандартном представлении
\n	Перевод строки; это не спецификатор, а управляющая последовательность

Несколько подстановок

```
fmt.Printf("Товар: %s, количество: %d\n", title, quantity)
```

Подстановки сопоставляются с аргументами слева направо. Для каждой должен существовать соответствующий аргумент подходящего типа.

Практический совет
Для простых сообщений используй Println. Когда нужен контролируемый формат или несколько значений внутри одной фразы, используй Printf.

9. Ввод данных и символ &

Программа может получать значения из стандартного ввода - обычно из терминала. Для первого урока используем fmt.Scan.

```
var quantity int
fmt.Println("Введите количество:")
count, err := fmt.Scan(&quantity)
```

Что происходит по шагам

1. Создается переменная quantity типа int с начальным значением 0.
2. Печатается приглашение для пользователя.
3. Scan ожидает ввод и пытается распознать целое число.
4. При успехе распознанное число записывается в quantity.
5. Scan возвращает количество прочитанных элементов и ошибку.

Зачем нужен &quantity;

Обычная передача quantity дала бы функции только текущее число. Чтобы Scan могла изменить саму переменную, ей передают адрес места хранения. Оператор & получает адрес переменной.

Пока достаточно этой модели
Переменная похожа на подписанную ячейку. quantity читает содержимое ячейки, а &quantity указывает, где ячейка находится. Указатели подробно изучим в отдельном уроке.

Что возвращает Scan

Переменная count показывает число успешно прочитанных аргументов. err равна nil при успехе или содержит описание проблемы. Если count не нужен, его можно явно проигнорировать с помощью _.

```
_, err := fmt.Scan(&quantity)
```

Нижнее подчеркивание называется пустым идентификатором. Оно показывает: значение возвращается, но в этой точке программы оно намеренно не используется.

10. Ошибка ввода и проверка значения

Ошибка чтения и недопустимое значение - разные ситуации. Их нужно проверять отдельно.

Ошибка операции

```
_, err := fmt.Scan(&quantity)
if err != nil {
fmt.Println("Не удалось прочитать целое число")
return
}
```

Если пользователь вводит слово вместо числа, Scan не может выполнить обещанную операцию. err становится не nil. Программа сообщает о проблеме и завершает main через return.

Ошибка предметного правила

```
if quantity < 1 || quantity > 10 {
fmt.Println("Количество должно быть от 1 до 10")
return
}
```

Число было прочитано успешно, но оно нарушает правило программы. Это валидация данных, а не ошибка Scan.

Ввод	Чтение	Валидация
abc	Ошибка: это не int	Не выполняется
0	Успех: это int	Ошибка: вне диапазона
5	Успех	Успех

Почему проверка идет именно в таком порядке

Нельзя надежно проверять диапазон, пока неизвестно, удалось ли вообще получить значение. Сначала проверяется результат операции, затем смысл полученных данных.

Полезная схема
Ввод -> проверка err -> проверка правил -> основная логика -> вывод результата. Эта последовательность подойдет обеим практическим программам.

11. Условие if

if выбирает, выполнять ли блок кода. Выражение после if должно иметь тип bool.

```
if temperature < 0 {
fmt.Println("На улице мороз")
}
```

if и else

```
if balance >= cost {
fmt.Println("Средств достаточно")
} else {
fmt.Println("Недостаточно средств")
}
```

Выполнится ровно одна из двух ветвей. else относится к ближайшему незавершенному if.

Несколько вариантов

```
if score >= 90 {
fmt.Println("Отлично")
} else if score >= 60 {
fmt.Println("Зачет")
} else {
fmt.Println("Нужно повторить тему")
}
```

Условия проверяются сверху вниз. После первой истинной ветви остальные не выполняются.

Логические операторы

Оператор	Смысл	Пример
&&	И: истинны обе части	age >= 18 && confirmed
||	ИЛИ: истинна хотя бы одна часть	role == "admin" || role == "owner"
!	НЕ: меняет true на false и наоборот	!blocked

Не путай
Один символ = означает присваивание. Два символа == означают сравнение на равенство.

12. Выбор через switch

switch удобен, когда одно выражение сравнивается с несколькими конкретными вариантами.

```
switch status {
case "new":
fmt.Println("Новый")
case "paid":
fmt.Println("Оплачен")
case "cancelled":
fmt.Println("Отменен")
default:
fmt.Println("Неизвестный статус")
}
```

Как выполняется switch

1. Вычисляется значение status.
2. Оно сравнивается с вариантами case сверху вниз.
3. Выполняется тело совпавшего case.
4. Если совпадений нет, выполняется default, когда он присутствует.

break обычно не нужен

После выполнения case Go автоматически выходит из switch. Переход в следующий case не происходит. Это отличается от поведения некоторых других языков.

Несколько значений в одной ветке

```
switch day {
case "Saturday", "Sunday":
fmt.Println("Выходной")
default:
fmt.Println("Рабочий день")
}
```

Когда выбирать if, а когда switch

- Используй if для диапазонов, сложных логических выражений и проверки ошибок.
- Используй switch для набора четких вариантов одного значения.

Для задания
Таблица соответствия «номинал -> цена» естественно выражается через switch. Конкретную реализацию и возвращаемые значения функции выбери самостоятельно.

13. Цикл for

Цикл повторяет блок кода. В Go есть одно ключевое слово цикла - for, но оно поддерживает несколько форм.

Цикл со счетчиком

```
for i := 1; i <= 5; i++ {
fmt.Println(i)
}
```

Часть	Назначение
i := 1	Выполняется один раз до начала; создается счетчик
i <= 5	Проверяется перед каждой итерацией
i++	Выполняется после каждой итерации
тело	Выполняется, пока условие истинно

Проследим значения

Перед первой итерацией i равно 1. После печати выполняется i++, и i становится 2. Когда после пятой итерации i станет 6, условие 6 <= 5 окажется ложным, и цикл завершится.

Цикл с одним условием

```
for balance > 0 {
balance--
}
```

Такой for повторяется, пока условие истинно. Если значение в условии никогда не меняется, цикл может стать бесконечным.

break и continue

- break немедленно завершает ближайший цикл.
- continue пропускает остаток текущей итерации и переходит к следующей.

Накопитель

```
sum := 0
for i := 1; i <= limit; i++ {
sum += i
}
```

sum существует до цикла и на каждой итерации хранит промежуточный итог. После завершения в ней остается окончательный результат.

14. Функции: параметры и результат

Функция объединяет действие под именем. Она помогает убрать повторение, отделить правила и сделать main понятнее.

```
func double(value int) int {
result := value * 2
return result
}
```

Разбор объявления

- func сообщает, что объявляется функция.
- double - имя, по которому функцию вызывают.
- value int - параметр: внутри функции будет доступно имя value типа int.
- Последний int - тип возвращаемого результата.
- return result завершает функцию и передает значение вызывающему коду.

Вызов

```
answer := double(4)
fmt.Println(answer)
```

Число 4 является аргументом. При вызове оно становится значением параметра value. Функция возвращает 8, и это значение записывается в answer.

Функция без результата

```
func printGreeting(name string) {
fmt.Println("Здравствуйте,", name)
}
```

После скобок параметров нет типа результата. Функция выполняет действие, но не передает вычисленное значение назад.

Хорошая граница функции

Имя и сигнатура должны отражать одну понятную ответственность. Функцию легче проверить, если она получает необходимые данные через параметры и возвращает результат, а не скрыто зависит от множества внешних переменных.

15. Несколько результатов и область видимости

Несколько возвращаемых значений

```
func divide(a int, b int) (int, bool) {
if b == 0 {
return 0, false
}
return a / b, true
}
```

Функция возвращает результат деления и признак успеха. При нулевом делителе число 0 само по себе неоднозначно, поэтому bool отдельно сообщает, допустима ли операция.

```
result, ok := divide(10, 2)
if !ok {
fmt.Println("Деление невозможно")
return
}
fmt.Println(result)
```

Сигнатура

Сигнатура описывает входы и выходы функции: имя не всегда считают ее частью в формальном определении, но в практике под выбором сигнатуры часто имеют в виду параметры и возвращаемые типы. Для priceForNominal тебе нужно решить, какие данные она получает и как сообщает об успехе.

Область видимости

```
func main() {
value := 10
if value > 5 {
message := "Большое значение"
fmt.Println(message)
}
// Здесь message уже недоступна.
}
```

Имя доступно от места объявления до конца ближайшего блока фигурных скобок. Внешний блок виден внутреннему, но внутренний не виден внешнему.

Экспортируемые имена

На уровне пакета имя с заглавной буквы доступно другим пакетам, например fmt.Println. Имя со строчной буквы остается внутренним. В текущих однопакетных заданиях функции разумно назвать со строчной буквы.

16. Как хранить деньги

Денежные расчеты требуют точности. Типы float32 и float64 хранят двоичные приближения дробей. Некоторые привычные десятичные значения нельзя представить ими точно.

Учебная модель

В заданиях все цены заданы в целых рублях, поэтому можно использовать int. Например, 1090 означает 1090 рублей.

Модель реального сервиса

Если нужны копейки, сумму обычно хранят в минимальных единицах валюты как целое число. 1090 рублей 50 копеек превращаются в 109050 копеек. Отображение с точкой или запятой выполняется только на границе интерфейса.

Ситуация	Представление
Учебная цена без копеек	int: 1090 рублей
Реальная сумма с копейками	int64: 109050 копеек
Процентная скидка	Явное правило округления после целочисленного расчета

Переполнение

У каждого целочисленного типа есть предел. Для небольшого учебного калькулятора int достаточен. В платежных системах часто выбирают int64 и отдельно контролируют допустимые диапазоны.

Итог не должен быть отрицательным

Если скидка теоретически может превысить стоимость, бизнес-правило должно ограничить результат нулем или отклонить операцию. Нельзя надеяться, что такой случай никогда не возникнет.

На будущее
Способ хранения денег, валюта, правила округления и налоги являются частью модели предметной области. Их фиксируют явно и покрывают тестами.

17. Как разбираться с ошибками

Не меняй код наугад

Сначала опиши ожидаемое поведение, затем фактическое. После этого найди первую точку, где значения расходятся.

Алгоритм диагностики

1. Воспроизведи проблему на самом маленьком вводе.
2. Прочитай первое сообщение компилятора полностью.
3. Открой указанную строку, но проверь и строку выше: незакрытая скобка часто проявляется позже.
4. Для логической ошибки выпиши значения переменных по итерациям.
5. Добавь временный Printf в точке, где значение меняется.
6. После исправления удали диагностический вывод и повтори все сценарии.

Трассировка цикла вручную

Итерация	i до тела	sum до	sum после
1	1	0	1
2	2	1	3
3	3	3	6

Такая таблица помогает увидеть неправильную начальную величину, границу цикла или порядок обновления.

Полезные команды

```
gofmt -w .
go vet ./...
go build ./...
```

gofmt исправляет форму кода. go vet ищет подозрительные конструкции. go build подтверждает, что программа компилируется. Ни одна из этих команд не гарантирует правильность бизнес-логики - ее проверяют сценариями и позднее тестами.

Практика 1. Анализатор чисел

Рабочий файл: non_project_tasks/lesson_01/number_analyzer/main.go

Требования

1. Запросить у пользователя целое число N.
2. Проверить ошибку чтения. При ошибке вывести понятное сообщение и завершить программу.
3. Проверить диапазон от 1 до 10 000 включительно.
4. Циклом пройти все целые числа от 1 до N включительно.
5. Накопить сумму всех чисел.
6. Посчитать количество четных и нечетных чисел.
7. Накопить сумму чисел, которые делятся на 3 без остатка.
8. Вывести результаты с понятными подписями.

Ограничение

Общую сумму нужно вычислить циклом. Математическую формулу суммы арифметической прогрессии использовать нельзя: цель задания - научиться управлять циклом и накопителями.

Контрольные случаи

Ввод	Ожидаемая проверка
1	Сумма 1; четных 0; нечетных 1; сумма кратных 3 равна 0
6	Сумма 21; четных 3; нечетных 3; сумма кратных 3 равна 9
0	Сообщение о недопустимом диапазоне
10001	Сообщение о недопустимом диапазоне
abc	Сообщение об ошибке чтения

Перед написанием кода

- Выпиши, какие переменные нужны и что хранит каждая.
- Определи начальные значения накопителей.
- Сформулируй условие продолжения цикла.
- Проследи вручную N = 3.

Практика 2. Калькулятор заказа

Рабочий файл: non_project_tasks/lesson_01/order_calculator/main.go

Это учебная CLI-программа. Она не является частью будущего production-сервиса и использует условные цены.

Номинал	Цена одной карты
1000	1090 рублей
2500	2690 рублей
5000	5290 рублей

Правила

- Прочитать номинал, количество и промокод; проверить ошибки чтения.
- Отклонить номинал, которого нет в таблице.
- Разрешить количество от 1 до 10 включительно.
- STARTGO дает скидку 100 рублей, если сумма до скидки не меньше 5000.
- Итоговая сумма не может быть отрицательной.
- Деньги хранить в целых рублях, не использовать float32 и float64.

Обязательная декомпозиция

Создай функции priceForNominal и calculateDiscount. Самостоятельно выбери параметры и возвращаемые значения. priceForNominal должна сообщать не только цену, но и то, поддержан ли номинал.

Итоговый вывод

Номинал, количество, цена одной карты, сумма до скидки, скидка и итог к оплате.

Сценарии

- 1000, количество 2, промокод отсутствует.
- 2500, количество 2, промокод STARTGO.
- 777, количество 1.
- Поддерживаемый номинал, количество 0 или 11.
- Строка вместо номинала.

Граница помощи
Если застрянешь, сформулируй: какой результат ожидался, что получилось, какой фрагмент непонятен и что уже проверено. Я дам направление, но не готовое решение.

18. Git: три состояния файлов

Git сохраняет историю проекта как последовательность снимков. Коммит не создается автоматически после изменения файла.

Область	Что там находится	Команда просмотра
Working tree	Текущее содержимое файлов на диске	git diff
Staging area	Изменения, выбранные для следующего коммита	git diff --staged
Repository	Уже сохраненные коммиты	git log --oneline

Working tree

Когда ты редактируешь main.go, меняется рабочая директория. Git видит отличие от последнего коммита, но еще не знает, нужно ли включать его в следующий снимок.

Staging area

Команда git add копирует выбранную версию изменения в индекс. Это подготовленный состав следующего коммита. После дальнейшего редактирования файла рабочая версия может отличаться от уже подготовленной.

Commit

git commit создает новый снимок именно из staging area. Он не берет произвольно все изменения рабочей директории.

Почему нужен git diff --staged
Эта команда показывает точное содержимое будущего коммита. Проверять его полезно перед каждым commit.

Ветка

Ветка - подвижное имя, указывающее на последний коммит линии разработки. Отдельная учебная ветка позволяет выполнять урок, не смешивая работу с основной веткой main.

```
git switch -c lesson/01-basics
```

Параметр -c создает новую ветку и сразу переключает рабочую директорию на нее.

Git: полный рабочий цикл урока

1. Создать учебную ветку

```
git switch -c lesson/01-basics
```

2. Создать Go-модуль

```
go mod init github.com/Amaym0n/go-start
```

3. Во время работы смотреть изменения

```
git status
git diff
```

status дает сводку, diff показывает конкретные измененные строки.

4. Выполнить проверки

```
gofmt -w .
go vet ./...
go build ./...
```

5. Подготовить изменения

```
git add .
git diff --staged
```

Точка означает текущую директорию. Перед add убедись, что находишься в корне проекта и понимаешь, какие файлы будут добавлены.

6. Создать коммит

```
git commit -m "Complete lesson 1 Go basics"
```

Сообщение отвечает на вопрос, что изменяет коммит. Оно должно быть коротким и конкретным.

7. Отправить ветку

```
git push -u origin lesson/01-basics
```

origin - имя удаленного репозитория. -u связывает локальную ветку с удаленной, чтобы следующие push и pull могли обходиться без явного имени ветки.

19. Проверка понимания

Ответь своими словами. Если ответ сводится к повторению одной фразы из текста, приведи маленький пример.

1. Чем модуль отличается от пакета?
2. Почему package должен находиться в начале каждого Go-файла?
3. Чем объявление функции отличается от ее вызова?
4. В чем разница между := и =?
5. Какие нулевые значения имеют int, string и bool?
6. Почему после fmt.Scan сначала проверяют err, а затем диапазон числа?
7. Что хранится в переменной типа bool?
8. Когда switch читается понятнее, чем цепочка if?
9. Из каких трех частей состоит for со счетчиком?
10. Что происходит с локальной переменной после выхода из ее блока?
11. Зачем функции возвращать пару (значение, bool)?
12. Почему для денег не следует бездумно использовать float64?

Вопросы по Git

13. Чем working tree отличается от staging area?
14. Почему git add не создает коммит?
15. Зачем смотреть git diff --staged?
16. Что делает -u при первом push новой ветки?

Не обязательно отвечать мне на все 16 вопросов письменно
Они нужны для самопроверки. При сдаче обязательны четыре ответа по Git из git_practice.txt. По Go я проверю понимание через код и уточняющие вопросы.

Чек-лист завершения и словарь

Перед сдачей

- Обе программы самостоятельно реализованы в подготовленных main.go.
- Проверены успешные, граничные и ошибочные сценарии.
- gofmt, go vet и go build завершаются без ошибок.
- В коде нет неиспользуемых переменных и импортов.
- Деньги не хранятся в float32 или float64.
- Просмотрены git diff и git diff --staged.
- Создан коммит и ветка отправлена на GitHub.
- Подготовлены ответы на четыре вопроса из git_practice.txt.

Краткий словарь

Термин	Определение
Аргумент	Конкретное значение, переданное при вызове функции
Блок	Область кода между фигурными скобками
Валидация	Проверка, соответствует ли значение правилам программы
Итерация	Одно выполнение тела цикла
Компиляция	Преобразование исходного кода в исполняемую форму
Параметр	Именованное входное значение в объявлении функции
Пакет	Совместно компилируемая группа Go-файлов одной директории
Сигнатура	Параметры и возвращаемые типы функции
Тип	Набор допустимых значений и операций над ними

Как сдавать
Напиши «Готово», пришли короткий хеш из git rev-parse --short HEAD и ответы по Git. Я изучу код, запущу проверки и дам замечания. Исправления остаются твоей практикой.